

웹 프로그래밍 프로젝트 과제

컴퓨터공학부 202201488 손정혁

■서비스 개요 및 주요 기능

서비스 개요

- React와 node.js 기반의 '똥 피하기'웹 게임 및 통합 랭킹 서비스

주요 기능

- 웹 브라우저 기반 미니 게임 플레이

HTML5 Canvas API를 활용하여 게임 렌더링 루프를 구현. 좌, 우 방향키와 Space Bar 점프 키를 통해 하늘에서 내려오는 똥을 피해서 오래 버틸수록 높은 점수를 획득

- 사용자 인증 시스템

DBMS와 연동하여 DB에 회원들의 정보를 저장하고 이를 확인 함에 따른 게임 회원가입 및 로그인 기능을 제공 (로그인 시 게임 로비에 접근 가능)

- 통합 리더보드 제공

게임 종료 시 생존 시간 데이터를 백엔드(Node.js)로 전송하여 DB에 저장하고, 로비 화면에서 상위 유저와 자신의 랭킹조회를 가능하게 함

- 로컬 환경설정 유지

Web Storage(localStorage)를 활용하여 사용자의 BGM 켜기/끄기 설정 상태를 브라우저에 저장하고 유지하도록 하여 서버 부하를 최소화

■ 사용할 기술 스택

Frontend

- UI 및 상태 관리

React.js를 통해 생성한 App.jsx 파일을 통해 웹 페이지의 버튼과 입력 칸 같은 UI 및 현재 캐릭터의 위치와 떨어지는 공격물 생성 주기와 속도 등을 관리

- 게임 엔진 및 렌더링

HTML5 Canvas API, requestAnimationFrame

실시간으로 웹페이지 화면을 새로 그리면서 부드러운 캐릭터 움직임을 보여주기 위해서 HTML5 Canvas API와 requestAnimationFrame을 사용하여 화면 상에 필요한 이미지

- HTTP 통신

Axios

비동기 통신을 위해 Promise 기반의 HTTP 클라이언트 라이브러리인 Axios를 사용하여 에러가 발생 시 바로바로 에러 처리를 진행하고 백엔드 서버와의 데이터 교환 역할을 수행

Backend

- 서버 프레임워크

Node.js. Express.js

API 설계

RESTful API 아키텍처

클라이언트와 서버 간 통신을 위한 URL과 HTTP 메소드를 분리하도록 설계

URL만 봐서 어떤 자원의 요청인지 파악할 수 있도록 주요 엔드포인트 구성

ex) POST /api/auth/register

■ Web Storage 및 DBMS 사용 전략

DBMS

- PostgreSQL 사용
- 데이터 일관성과 안정적인 트랜잭션처리를 위해 RDBMS 채택. 사용자 계정 정보와 게임 최고 점수(생존 시간) 데이터를 테이블로 분리하여 영구적으로 안전하게 관리

Web Storage(localStorage)

- BGM과 같은 가벼운 클라이언트 상태 데이터는 서버에 매번 요청할 필요가 없으므로 웹 브라우저의 Web Storage에 저장하여 사용자 편의성 및 프론트엔드 성능을 향상

■ Reverse-proxy 및 Docker 사용 계획

Docker

- 프론트엔드(React)와 백엔드(Node.js) 애플리케이션을 각각 독립된 컨테이너로 규격화하여, 로컬 환경과 배포 환경 간의 일관성을 확보하고 종속성 충돌을 방지

Reverse-Proxy

- Nginx를 활용하여 정적 파일(React 빌드 결과물)을 클라이언트에 전달하고, 특정 경로(/api 등)의 요청은 Node.js 백엔드 컨테이너로 리버스 프록시 라우팅 처리를 하여 클라이언트가 직접적으로 백엔드 서버에 접속하는 것을 막게 설계

■ CI/CD 계획

- 버전 관리 및 파이프라인: GitHub & GitHub Actions
- 배포 자동화 전략: 메인 브랜치에 코드가 병합(Push)되면 GitHub Actions가 트리거되어 자동으로 Docker 이미지를 빌드하고, Render 클라우드 환경에 존재하는 Web project를 자동으로 Deploy 되도록 CI/CD 파이프라인 구축

■ Github Public Repository URL

https://github.com/yu990912/Web_Game_project.git